SIMATS ENGINEERING

Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA3714**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5
11	A smart thermostat's firmware execution trace shows a recurring pattern of "nested loops" during temperature calibration. Reverse engineer the white box testing strategy by identifying the loop invariants and termination conditions from the trace. Decompose the calibration module into its logical sub-routines. Reconstruct a path testing strategy specifically targeting the boundary values of the loops. Justify your approach in terms of preventing firmware hangs or infinite loops.	5
12	A third-party "Tax Calculation" DLL provides specific error codes for different income brackets but has no documentation. Reverse engineer the tax-slab logic by analyzing the output error codes against varied input ranges. Decompose the system into functional components based on income categories. Reconstruct a set of test cases using ECP and BVA. Provide a structured explanation of how your reconstructed test suite validates the correctness of the hidden tax laws.	5
13	An automated flight booking system's integration test reports show that errors only occur when "Frequent Flyer" status is combined with "International" travel and "Last-minute" booking. Reverse engineer the testing approach to identify why these multi-variable combinations were missed. Apply a combination of black box (Cause-Effect Graphing) and white box (Path Testing) to reconstruct a hybrid testing strategy. Justify how this approach ensures maximum defect detection in complex business workflows.	5

Code of Conduct:

I Mathusri P-192311363 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

QUESTION 1

Reverse Engineering of ATM Withdrawal Module

1. Understanding of the System

The ATM withdrawal module accepts a withdrawal amount and verifies whether the transaction is valid. From the observed input-output behaviour, the main rules can be reconstructed as:

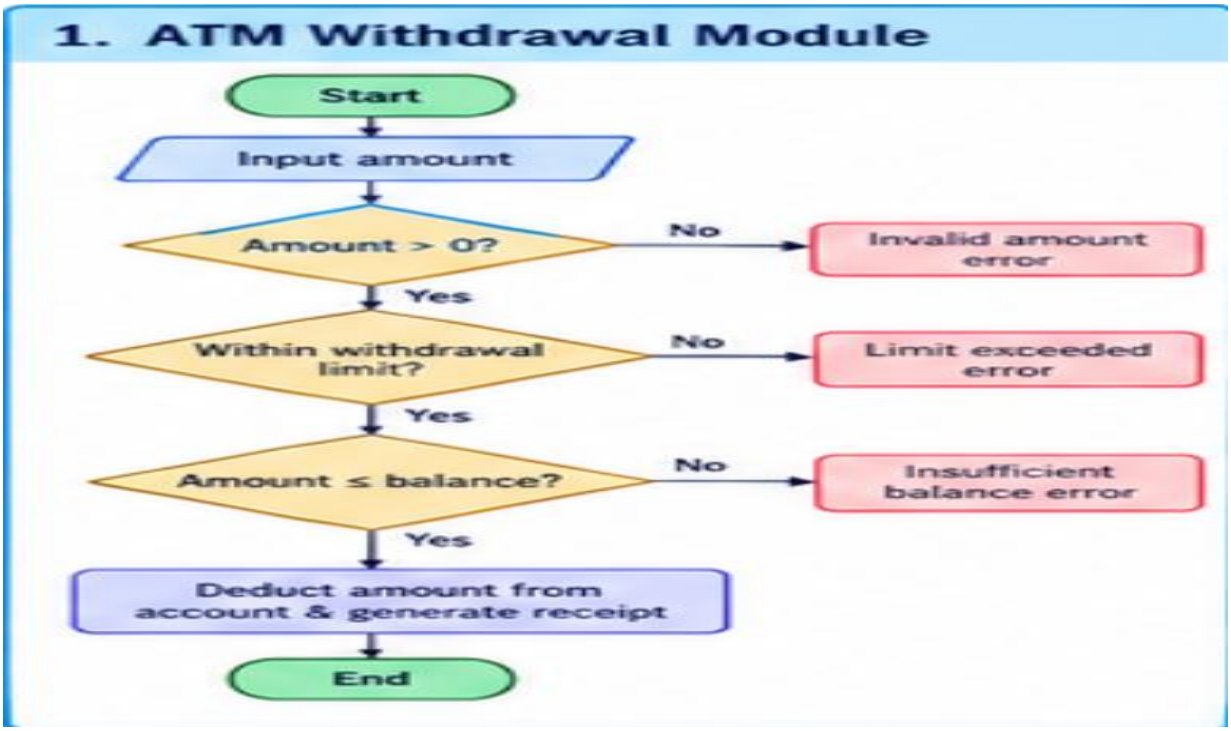
- The withdrawal amount must be positive.
- The amount must satisfy the ATM withdrawal limit.
- The requested amount must not exceed the available account balance.
- The transaction should display an appropriate error message for invalid input.
- A valid transaction reduces the account balance.

The likely original testing approach was **black-box testing**, because only input-output behaviour is available.

2. Functional Decomposition

The ATM withdrawal system can be divided into:

1. **Input Validation**
    - Checks whether the entered amount is valid.
2. **Withdrawal Limit Validation**
    - Checks minimum and maximum withdrawal limits.
3. **Balance Validation**
    - Compares requested amount with available balance.
4. **Transaction Processing**
    - Deducts the amount from the account.
5. **Error Handling**
    - Displays suitable error messages.
6. **Receipt/Result**
    - Displays successful or failed transaction information.



3. Equivalence Classes

Input Condition	Equivalence Class	Example
Amount ≤ 0	Invalid	0, -500
Valid withdrawal range	Valid	₹5,000
Amount above limit	Invalid	₹50,000
Amount ≤ balance	Valid	₹5,000 with ₹20,000 balance
Amount > balance	Invalid	₹25,000 with ₹20,000 balance
Non-supported denomination	Invalid	₹1,250 if only multiples of ₹100 are allowed

4. Boundary Value Analysis

Important boundary values include:

- Minimum withdrawal − 1
- Minimum withdrawal
- Minimum withdrawal + 1
- Maximum withdrawal − 1
- Maximum withdrawal
- Maximum withdrawal + 1
- Balance − 1
- Balance
- Balance + 1

5. Reconstructed Test Cases

TC	Input	Expected Result
TC01	Valid amount within limit and balance	Withdrawal successful
TC02	Amount = 0	Invalid amount error
TC03	Negative amount	Invalid amount error
TC04	Minimum − 1	Reject
TC05	Minimum amount	Accept
TC06	Maximum amount	Accept
TC07	Maximum + 1	Limit exceeded
TC08	Amount equal to balance	Accept
TC09	Amount greater than balance	Insufficient balance
TC10	Unsupported denomination	Invalid denomination

6. Adequacy Evaluation

The original testing appears to have used basic equivalence partitioning because normal and invalid inputs were considered. However, it may not adequately cover boundary conditions. The improved design combines:

- Equivalence Partitioning
- Boundary Value Analysis
- Error guessing

This provides better coverage of withdrawal limits, balance limits and invalid inputs.

Conclusion

The ATM module can be effectively reverse engineered using observed input-output behaviour. The improved black-box test suite covers normal, invalid and boundary conditions and reduces the possibility of withdrawal-limit and balance-validation defects.

QUESTION 2

Reverse Engineering White-Box Testing Using Control Flow Graph

1. Understanding of the System

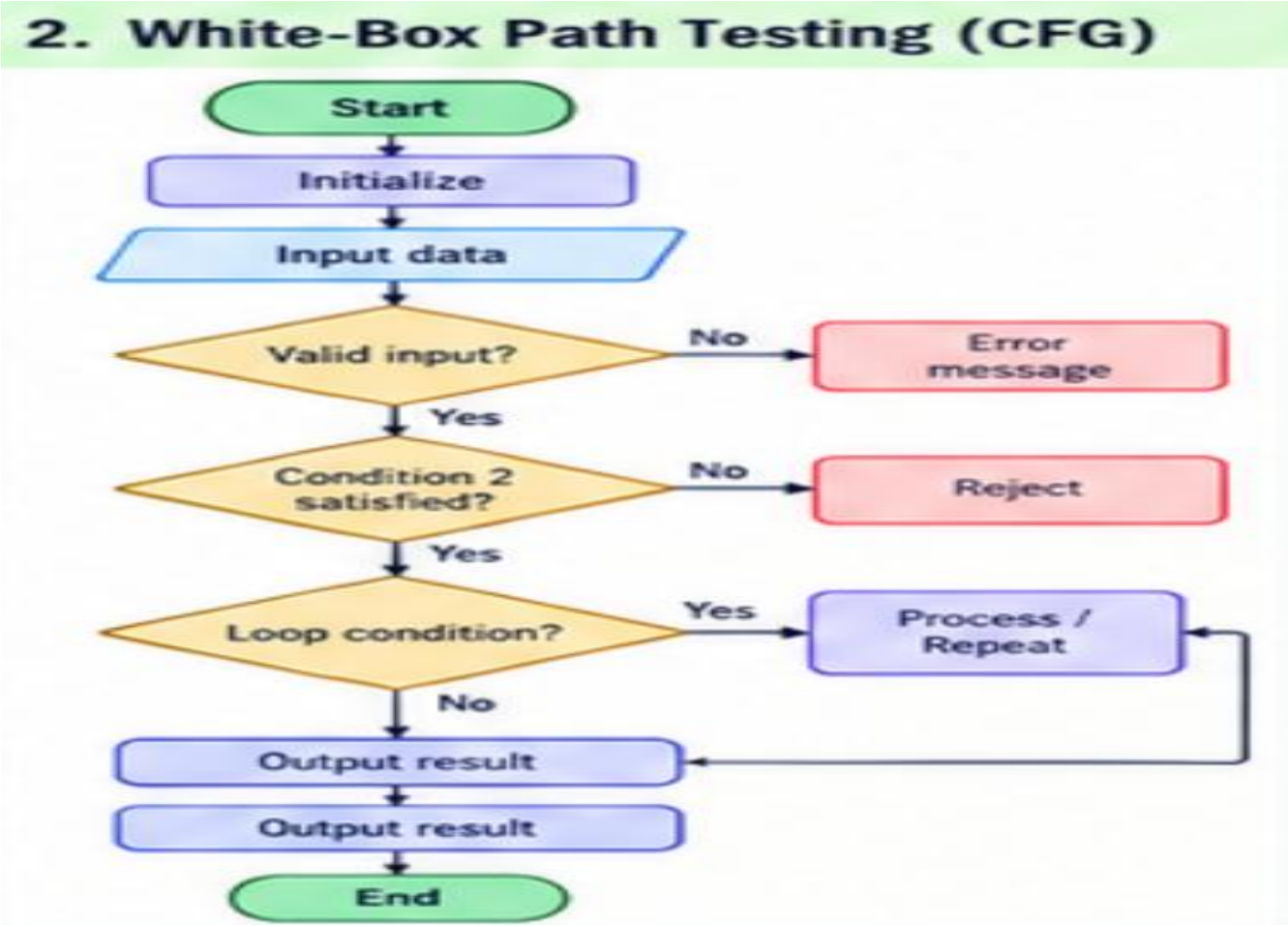
The given execution traces and code snippets provide internal implementation information. Therefore, the appropriate testing approach is **white-box testing**.

The objective is to identify:

- Statements
- Decisions
- Loops
- Branches
- Independent execution paths

2. Control Flow Representation

A conceptual control flow can be represented as:



3. Decision Points

Each condition creates a branch. For example:

- **Decision D1:** Is the input valid?
- **Decision D2:** Is the processing condition satisfied?
- **Decision D3:** Should the loop continue?

Each decision should be tested for both **True** and **False** outcomes.

4. Independent Paths

The paths should include:

- Path 1: Start → Input → Invalid → Error → End
- Path 2: Start → Input → Valid → Process → Output → End
- Path 3: Valid → Condition False → Reject → End
- Path 4: Valid → Condition True → Process → Loop → Output → End
- Path 5: Loop repeats before termination

5. Coverage Evaluation

The existing tests should be evaluated using:

- Statement coverage
- Branch coverage
- Condition coverage
- Path coverage
- Loop coverage

If a decision has only been tested as True, its False branch remains untested.

6. Reconstructed Path Testing Strategy

The improved strategy is:

1. Identify every node in the CFG.
2. Identify every decision node.
3. List independent paths.
4. Create at least one test case for every branch.
5. Execute loop zero, one and multiple iterations.
6. Verify expected output for each path.
7. Calculate statement and branch coverage.

Conclusion

White-box reverse engineering allows the internal execution structure to be reconstructed from code and traces. A CFG-based path strategy provides stronger coverage than simply testing expected outputs.

QUESTION 3

Reverse Engineering Login and Transaction System

1. System Understanding

The web application contains two major functional areas:

- User authentication
- Transaction processing

The defect logs can be used to infer the original testing techniques. Login defects involving invalid usernames, passwords and boundary inputs indicate **black-box testing**. Transaction defects involving specific execution branches indicate possible **white-box/path testing**.

2. Functional Decomposition

Web Application

```
|
├── Login
|   ├── Username validation
|   ├── Password validation
|   └── Authentication
|
├── Session Management
|   ├── Login session
|   └── Logout
|
└── Transaction
    ├── Input validation
    ├── Balance validation
    ├── Transaction processing
    └── Confirmation
```

3. Equivalence Partitioning

For username:

- Valid username
- Invalid username
- Empty username

For password:

- Valid password
- Invalid password

- Empty password

For transaction amount:

- Valid amount
- Zero/negative amount
- Amount greater than balance
- Amount above transaction limit

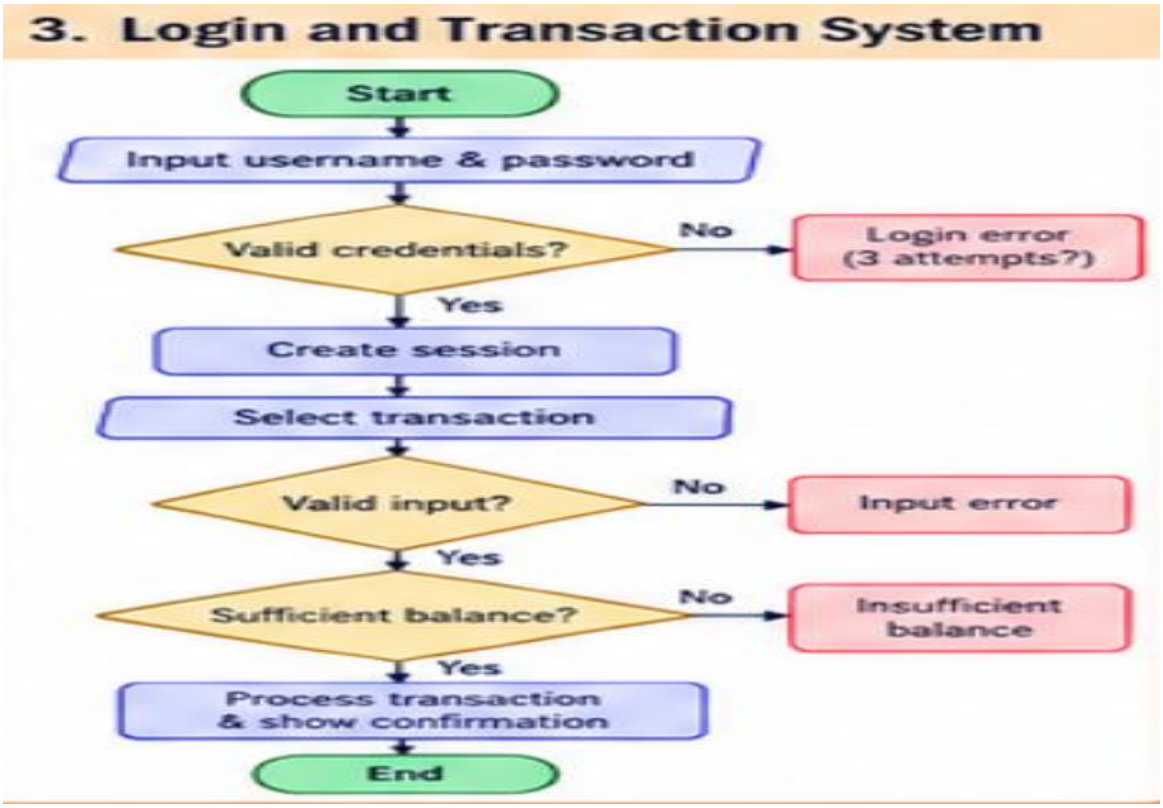
4. Boundary Values

Test:

- Minimum password length − 1
- Minimum password length
- Minimum password length + 1
- Maximum transaction − 1
- Maximum transaction
- Maximum transaction + 1
- Balance − 1
- Balance
- Balance + 1

5. Path Testing

Important paths include:



Both outcomes of every decision must be tested.

6. Coverage Gaps

Possible gaps include:

- Missing boundary cases
- Missing invalid combinations
- Uncovered transaction failure paths
- Insufficient branch testing
- No testing of repeated login attempts
- No testing of session expiration

Conclusion

A comprehensive strategy should integrate **ECP + BVA + Path Testing**. This combination provides input-level and code-level coverage and is more effective than relying on only one testing technique.

QUESTION 4

Reverse Engineering Legacy Inventory Management System

1. System Understanding

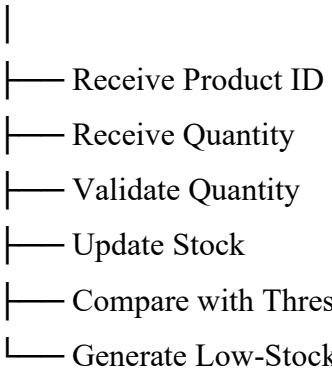
The inventory system processes stock updates and generates low-stock alerts. Successful and failed transaction logs reveal hidden business rules.

The likely rules are:

- Stock quantity cannot become negative.
- A valid stock update must follow permitted quantity rules.
- Low-stock alert is generated when stock reaches or falls below a defined threshold.
- Invalid quantities should be rejected.

2. Functional Decomposition

Inventory Update



3. Reconstructed Equivalence Classes

For stock quantity:

Class	**Condition**
EC1	Quantity < 0
EC2	Quantity = 0
EC3	Valid positive quantity
EC4	Quantity causing stock below zero

For low-stock condition:

Class	**Condition**
EC5	Stock > threshold
EC6	Stock = threshold
EC7	Stock < threshold

4. Boundary Conditions

If the low-stock threshold is **T**, test:

- T − 1
- T
- T + 1

For available stock **S**, test:

- S − 1
- S
- S + 1

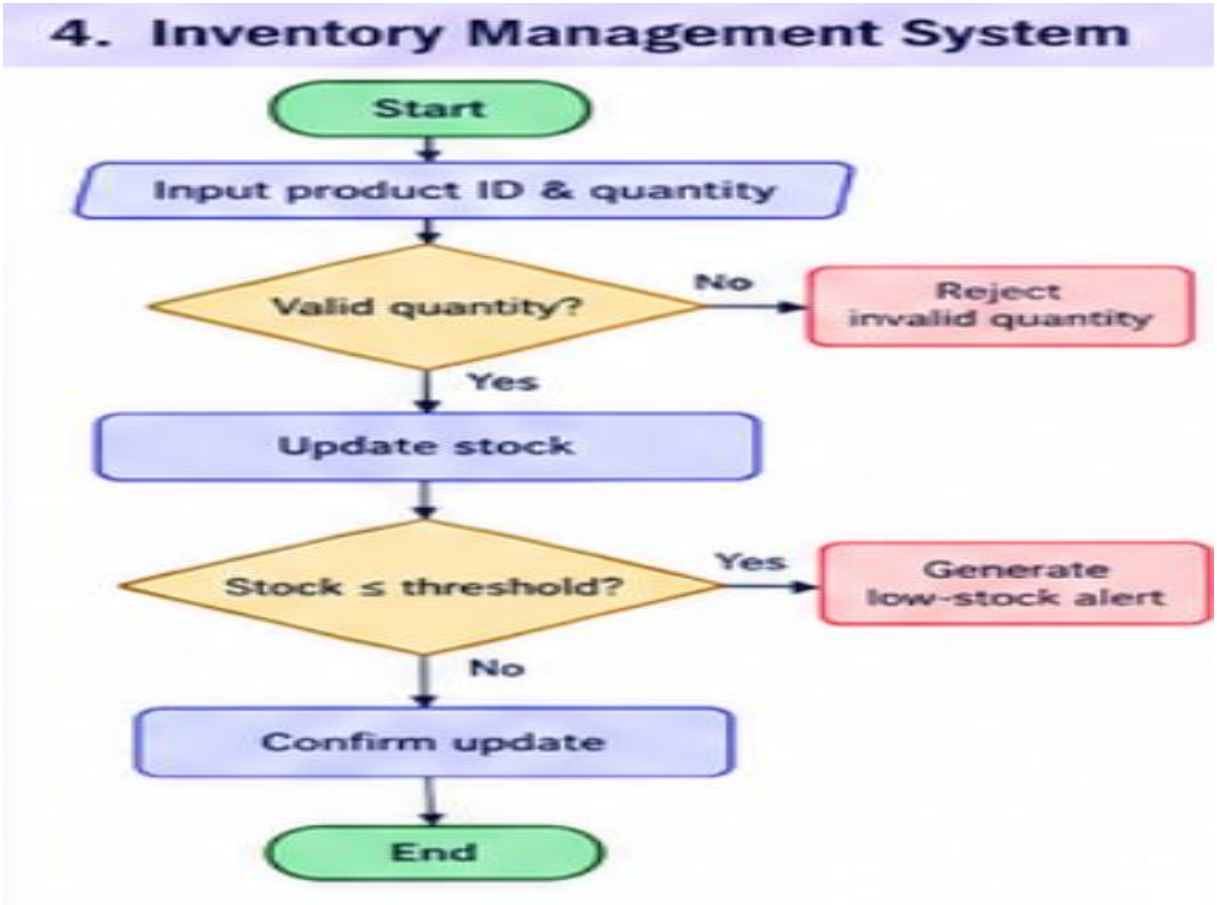
5. Reconstructed Test Cases

Test	Condition	Expected Result
TC01	Valid positive update	Stock updated
TC02	Quantity = 0	Apply defined zero rule
TC03	Negative quantity	Reject
TC04	Stock remains above threshold	No alert
TC05	Stock = threshold	Alert
TC06	Stock below threshold	Alert
TC07	Update makes stock negative	Reject

6. Testing Gaps

The logs may fail to cover:

- Exact threshold
- Just above threshold
- Just below threshold
- Zero quantity
- Negative values
- Large quantities
- Repeated updates



Conclusion

The hidden inventory logic can be reconstructed using ECP and BVA. The improved black-box test design specifically targets the boundary conditions revealed by the transaction logs.

QUESTION 5

Reverse Engineering Library Management Module Using Execution Profile

1. System Understanding

The compiled .exe does not provide direct source-code visibility. However, CPU and memory execution profiles can reveal resource-intensive sections.

A sudden CPU spike may indicate:

- Loop execution
- Repeated searching
- Sorting
- Large data processing

A memory spike may indicate:

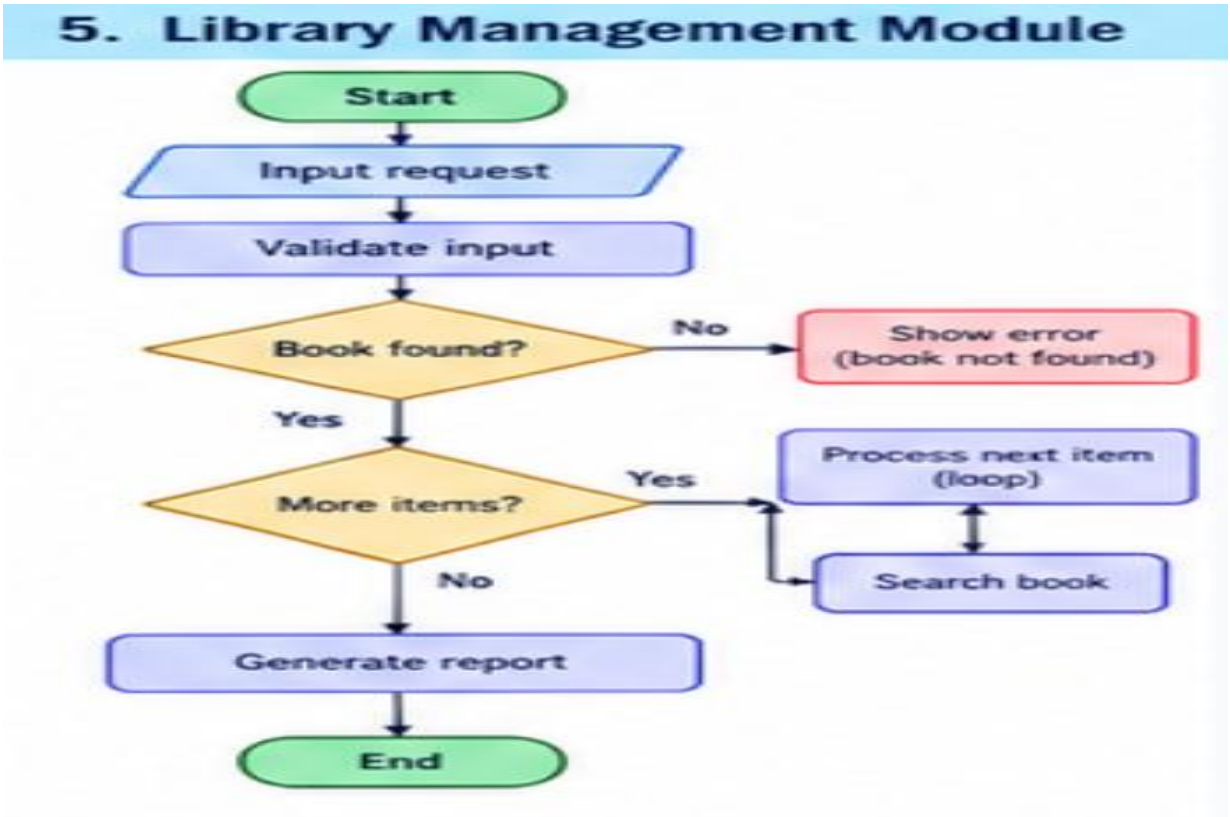
- Large data structures
- Object creation inside loops
- Unreleased memory
- Recursive processing

2. Functional Decomposition

The conceptual library module can be divided into:

1. Input processing
2. Book search
3. Book validation
4. Transaction processing
5. Database/file access
6. Report generation

3. Conceptual CFG



4. Resource-Intensive Paths

The most important path is:

Input → Search → Book Found → Process → More Items → Search

because repeated loop execution can cause CPU consumption.

A large-data path should also be tested to observe memory usage.

5. Path Testing Strategy

Test:

- Empty input
- One item
- Multiple items
- Maximum expected items
- Book found
- Book not found
- Loop zero times
- Loop once
- Loop many times
- Invalid input

6. Stability Evaluation

The reconstructed testing should monitor:

- CPU utilization
- Memory consumption
- Execution time
- Loop termination
- Application responsiveness

Resource-intensive paths should be executed repeatedly to determine whether performance degrades.

Conclusion

Although source code is unavailable, execution profiles provide useful evidence for reconstructing probable loops and decision clusters. Path testing focused on high-resource paths can expose stability and performance problems.

QUESTION 6

Reverse Engineering Mobile Payment State Machine

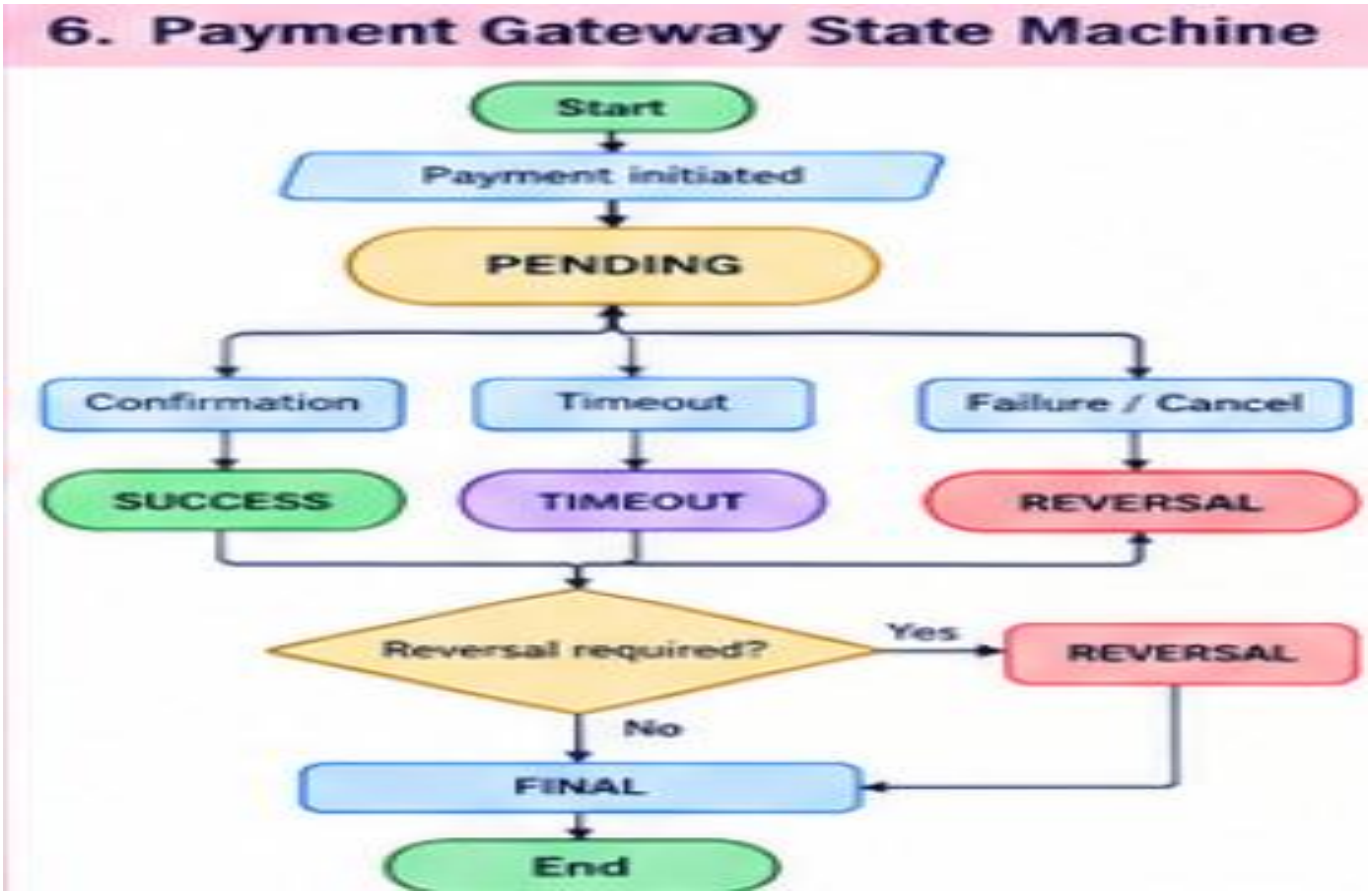
1. System Understanding

The payment gateway contains the following observable states:

- Pending
- Success
- Reversal
- Timeout

The API responses provide information about state transitions and constraints.

2. Reconstructed State Model



A successful payment may also require a reversal if a later failure condition occurs.

3. State Transition Testing

The implicit technique is **State Transition Testing** because the output depends on the current state and the event causing the transition.

4. Legal Transitions

Examples:

Current State	Event	Next State
Pending	Payment confirmed	Success
Pending	Timeout occurs	Timeout
Success	Valid reversal request	Reversal
Pending	Cancellation/failure	Reversal

5. Illegal Transitions

Examples:

- Success → Pending without a new transaction
- Timeout → Success without re-initiation
- Reversal → Success directly
- Reversal → Pending without a new transaction

These should be tested to verify that the system rejects invalid state changes.

6. Test Suite

TC	Current State	Event	Expected
TC01	Pending	Confirmation	Success
TC02	Pending	Timeout	Timeout
TC03	Pending	Failure	Reversal
TC04	Success	Valid reversal	Reversal
TC05	Success	Invalid pending request	Reject
TC06	Timeout	Success event	Reject
TC07	Reversal	Success event	Reject

Conclusion

State-transition testing provides systematic coverage of both legal and illegal payment transitions and is suitable for identifying incorrect transaction-state handling.

QUESTION 7

Reverse Engineering Encrypted Communication Module

1. System Understanding

The module accepts input parameters and returns encrypted outputs. Since the internal implementation is unavailable, testing begins with black-box fuzzing.

The system can be divided into:

1. Input validation
2. Data-size validation
3. Character/content validation
4. Encryption processing
5. Error handling
6. Output generation

2. Fuzzing Analysis

Different input types should be supplied:

- Empty input
- Normal input
- Maximum-size input
- Input exceeding maximum size
- Special characters
- Very long strings
- Unexpected data types
- Repeated characters
- Boundary-length strings

The purpose is to observe whether the system accepts, rejects or crashes.

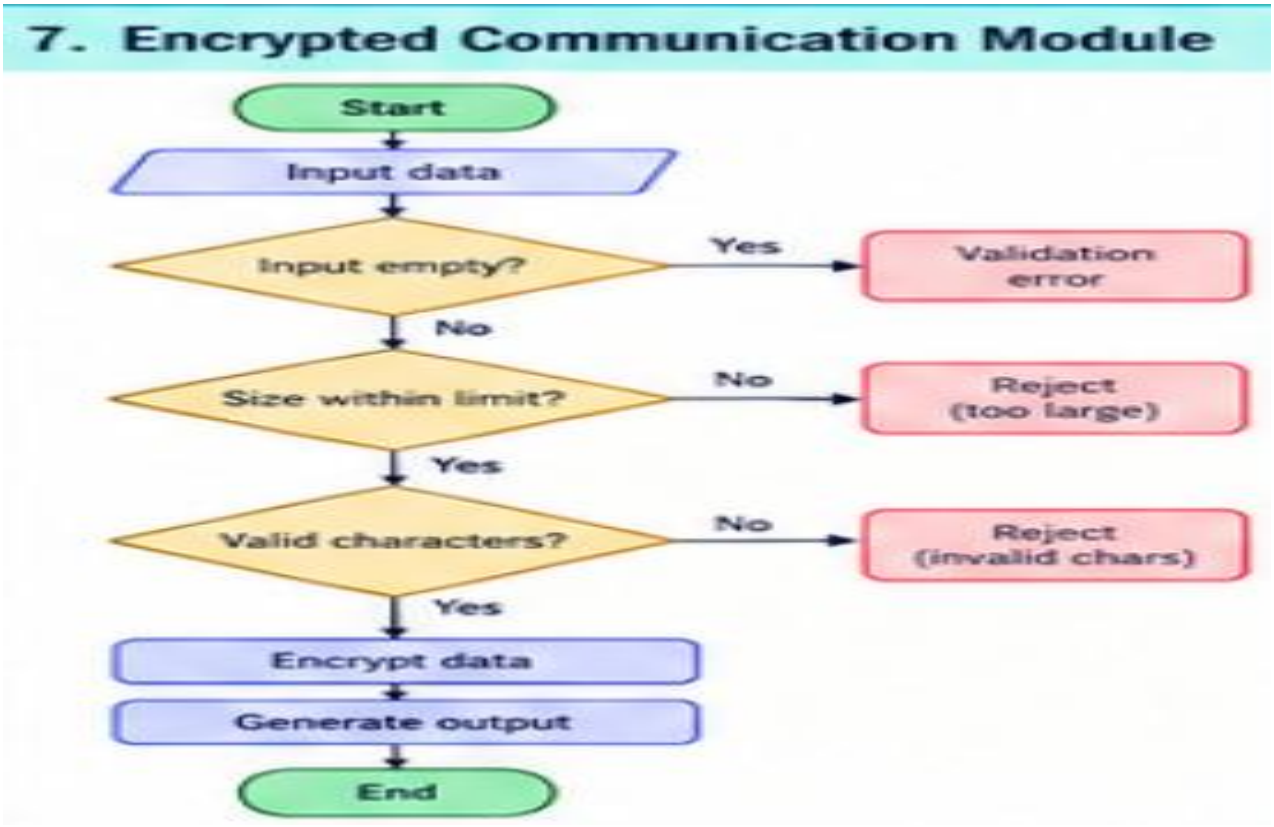
3. Boundary Value Analysis

If the permitted input size is **N**, test:

- N − 1
- N
- N + 1

Also test:

- 0
- 1
- Maximum valid size
- Maximum + 1



4. Reconstructed Test Cases

TC	Input	Expected
TC01	Empty input	Proper validation error
TC02	Minimum valid input	Accept
TC03	Normal input	Encrypt
TC04	Maximum valid input	Accept
TC05	Maximum + 1	Reject safely
TC06	Special characters	Validate safely
TC07	Very large input	Reject/handle safely

5. Security Considerations

The testing should ensure that oversized inputs do not cause:

- Buffer overflow
- Memory corruption
- Application crash

Unexpected characters should also be validated so that malicious input cannot alter processing behaviour.

Conclusion

Combining fuzzing with BVA provides effective black-box validation of hidden input boundaries. Safe rejection of invalid and oversized input reduces the risk of overflow and injection-related vulnerabilities.

QUESTION 8

Reverse Engineering Insurance Premium Calculator

1. Understanding of Existing Testing

The system has **100% statement coverage**, but defects remain in multi-condition if statements.

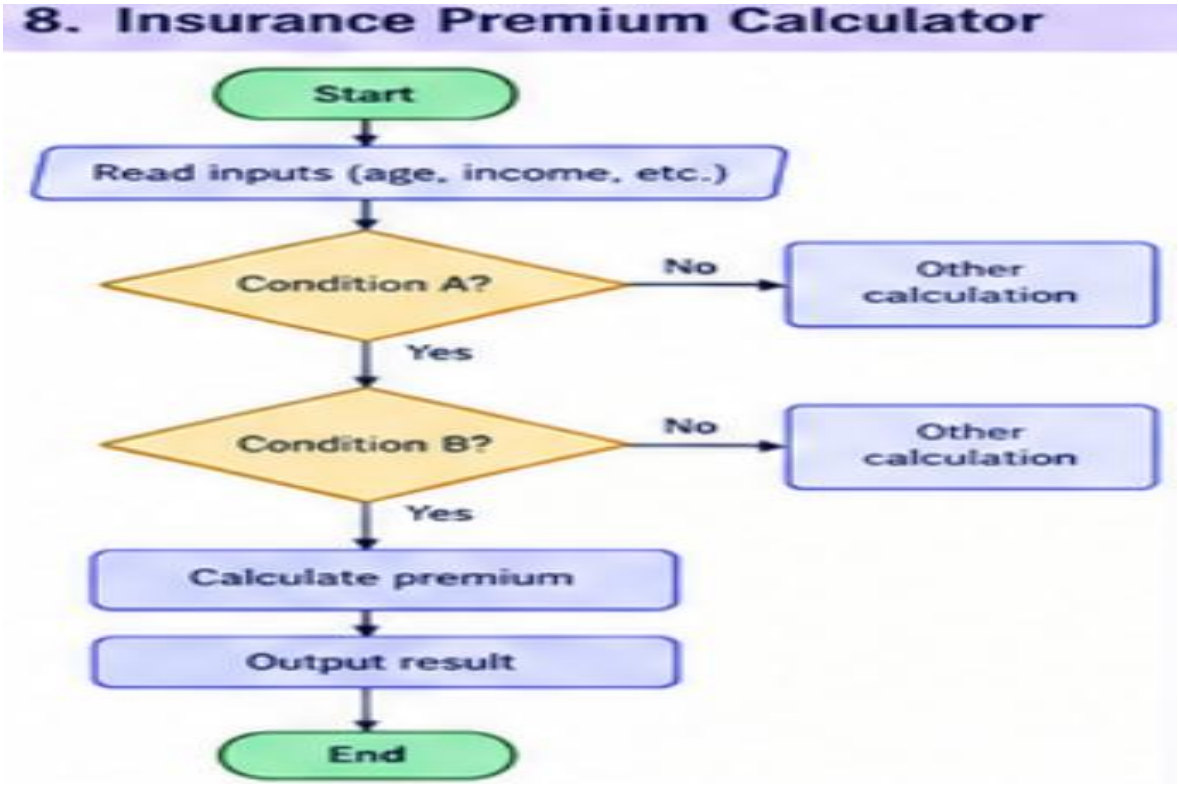
This shows that statement coverage alone is inadequate.

For example:

IF (Age > 50 AND Income > Limit)

A test may execute the statement without testing every possible Boolean combination.

2. Reconstructed CFG



3. Decision Conditions

For two Boolean conditions A and B, the combinations are:

A	B	Expected Branch
T	T	Main condition
T	F	Alternate
F	T	Alternate
F	F	Alternate

All combinations must be executed.

4. Independent Paths

- P1: A=True → B=True
- P2: A=True → B=False
- P3: A=False → B=True
- P4: A=False → B=False

5. Reconstructed Testing Strategy

The improved strategy must include:

- Statement coverage
- Branch coverage
- Condition coverage
- Path coverage

Boundary values should also be applied to age, income and other numerical inputs.

6. Benefit

The original 100% statement coverage only proves that statements executed. It does not guarantee that every decision outcome was exercised.

Branch and path testing exposes errors in:

- Premium calculation
- Multi-condition decisions
- Boundary conditions
- Incorrect logical operators

Conclusion

The major weakness of the original strategy was relying only on statement coverage. Achieving branch and independent-path coverage provides stronger confidence in the correctness of multi-condition premium calculations.

QUESTION 9

Reverse Engineering Cloud File Synchronization

1. System Understanding

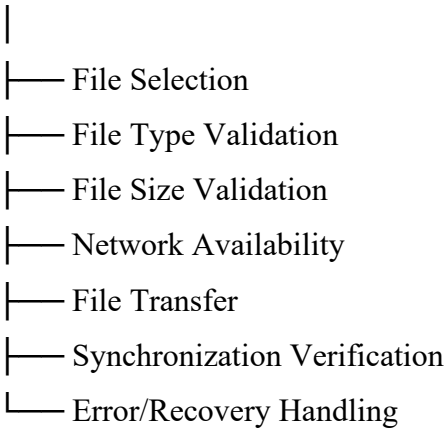
The synchronization system transfers files between locations through a network. Defect reports show that failures are related to:

- File size
- File type
- Network speed

Therefore, these variables are important for reconstructing the original ECP.

2. Functional Decomposition

File Synchronization



3. Equivalence Classes

File Size

- Empty/small file
- Normal file
- Maximum supported file
- Oversized file

File Type

- Supported type
- Unsupported type
- Corrupted file

Network Speed

- Stable/high speed
- Normal speed
- Low speed
- Intermittent connection

4. Boundary Value Analysis

For maximum file size **M**:

- M − 1
- M
- M + 1

For any supported file-count limit **F**:

- F − 1
- F
- F + 1

5. Stress Testing

The system should be tested with:

- Many simultaneous files
- Large files
- Multiple users
- Slow networks
- Repeated synchronization
- Interrupted transfers

6. Revised Strategy

ECP

+

BVA

+

Stress Testing

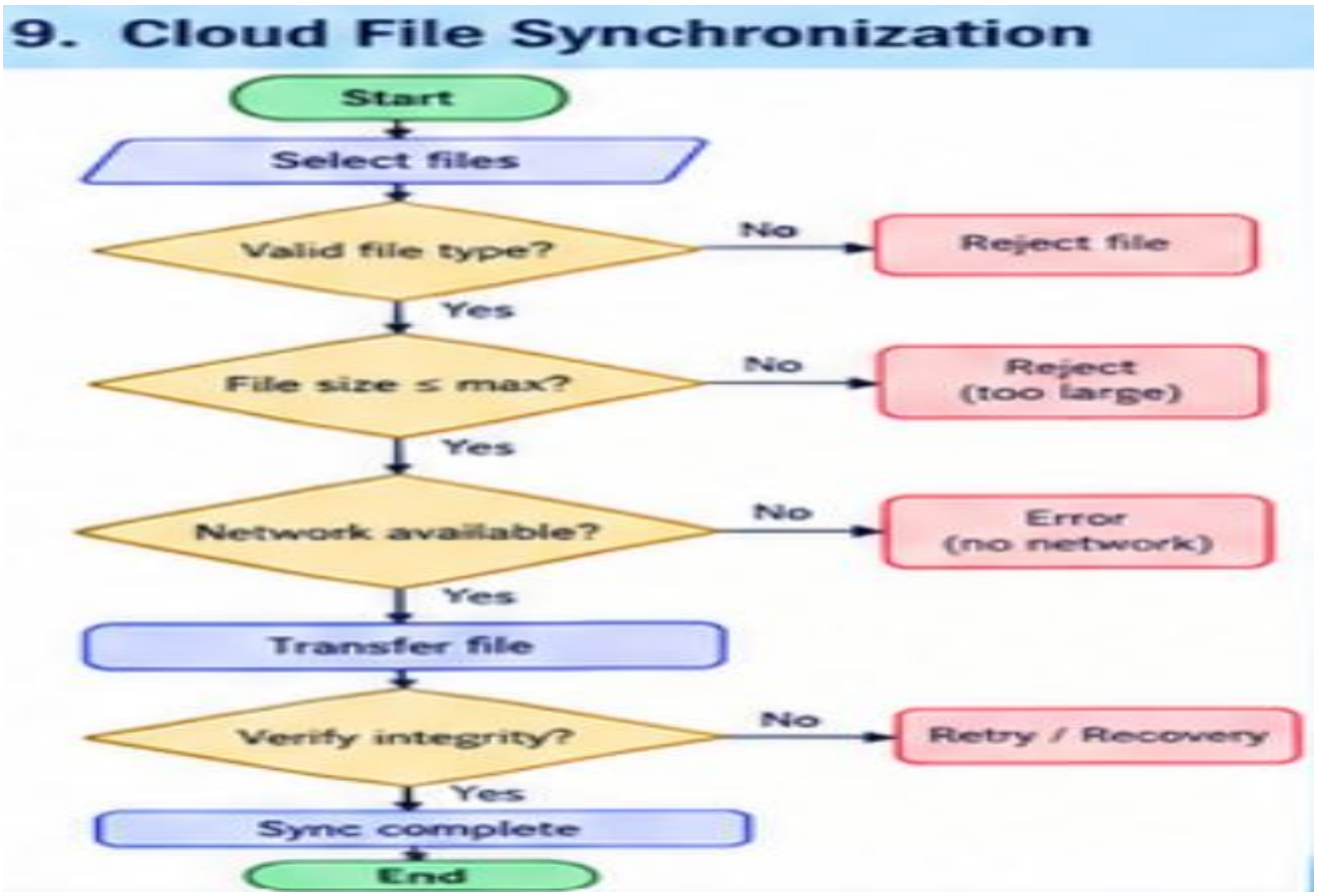
+

Recovery Testing

=

Improved Synchronization Testing

The system should verify file integrity after every transfer.



Conclusion

The reconstructed strategy focuses on the defect patterns identified in the logs. Combining ECP, BVA and stress testing can detect failures caused by extreme file sizes, unsupported types and poor network conditions and reduce future data loss.

QUESTION 10

Reverse Engineering E-Commerce Search and Filter Module

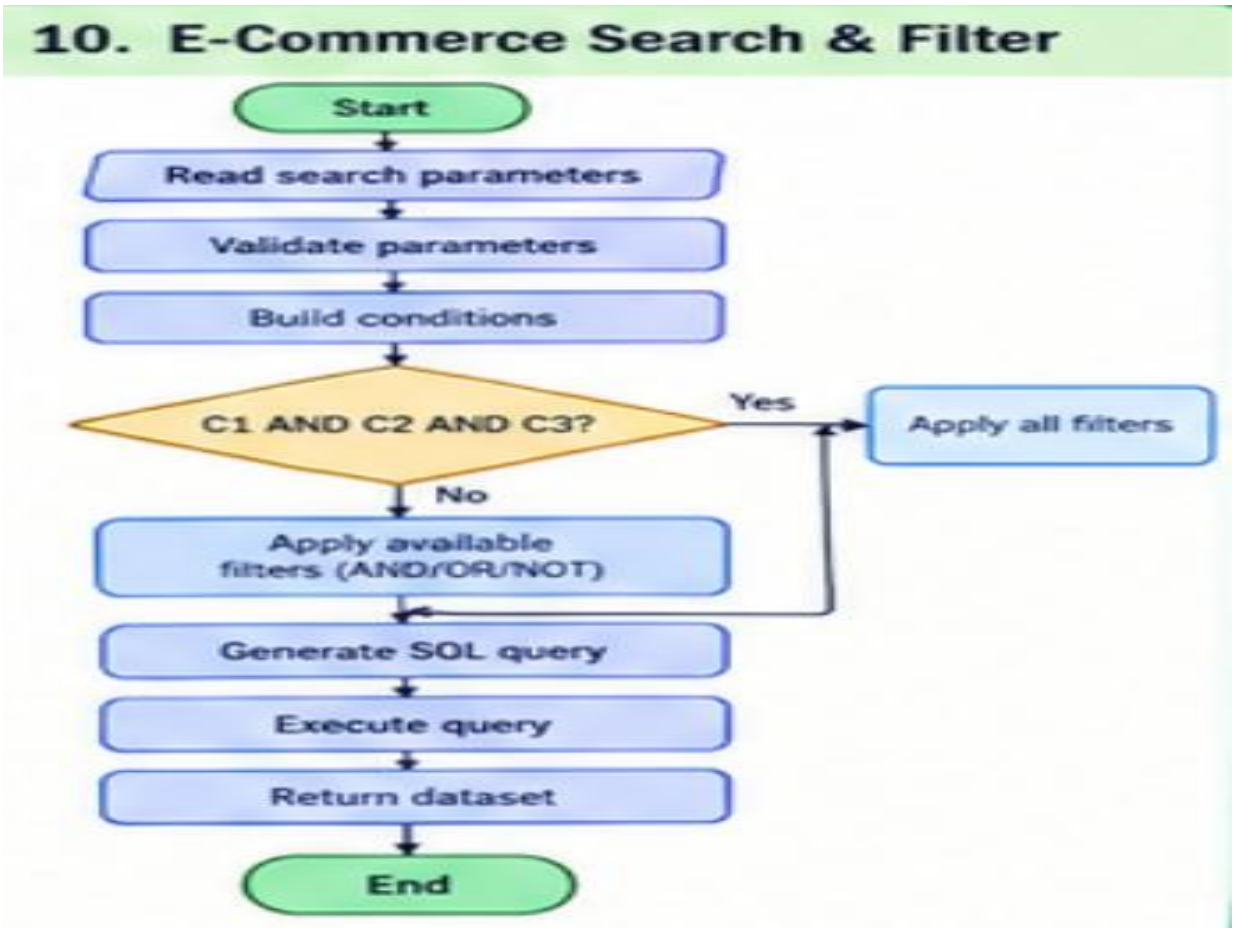
1. System Understanding

The module generates SQL queries based on search and filter parameters. The main decision logic involves:

- AND
- OR
- NOT

The resulting dataset can be compared with the input conditions to reconstruct the filtering logic.

2. Functional Decomposition



3. Decision Table

Consider three conditions:

- C1 = Category selected
- C2 = Price range selected
- C3 = Brand selected

Rule	C1	C2	C3	Expected
R1	Y	Y	Y	All conditions applied
R2	Y	Y	N	Category + Price
R3	Y	N	Y	Category + Brand
R4	N	Y	Y	Price + Brand
R5	Y	N	N	Category only
R6	N	Y	N	Price only
R7	N	N	Y	Brand only
R8	N	N	N	No filter / all valid results

4. AND/OR/NOT Testing

Examples:

- Category AND Brand
- Category OR Brand
- Category AND NOT Brand
- Price AND Category
- Multiple conditions together

5. Test Cases

TC	Filter	Expected
TC01	Category only	Matching category
TC02	Brand only	Matching brand
TC03	Category AND Brand	Both conditions satisfied
TC04	Category OR Brand	Either condition satisfied
TC05	NOT Brand	Brand excluded
TC06	All filters	All constraints satisfied
TC07	No filters	Default result

6. Completeness

For three independent binary conditions, the decision table contains **$2^3 = 8$ combinations**. Testing all combinations provides complete logical coverage for these three conditions.

Conclusion

Decision-table testing is suitable because the output depends on combinations of multiple filtering conditions. The reconstructed table systematically covers the possible combinations and helps detect incorrect AND/OR/NOT logic.

QUESTION 11

Reverse Engineering Smart Thermostat Nested Loops

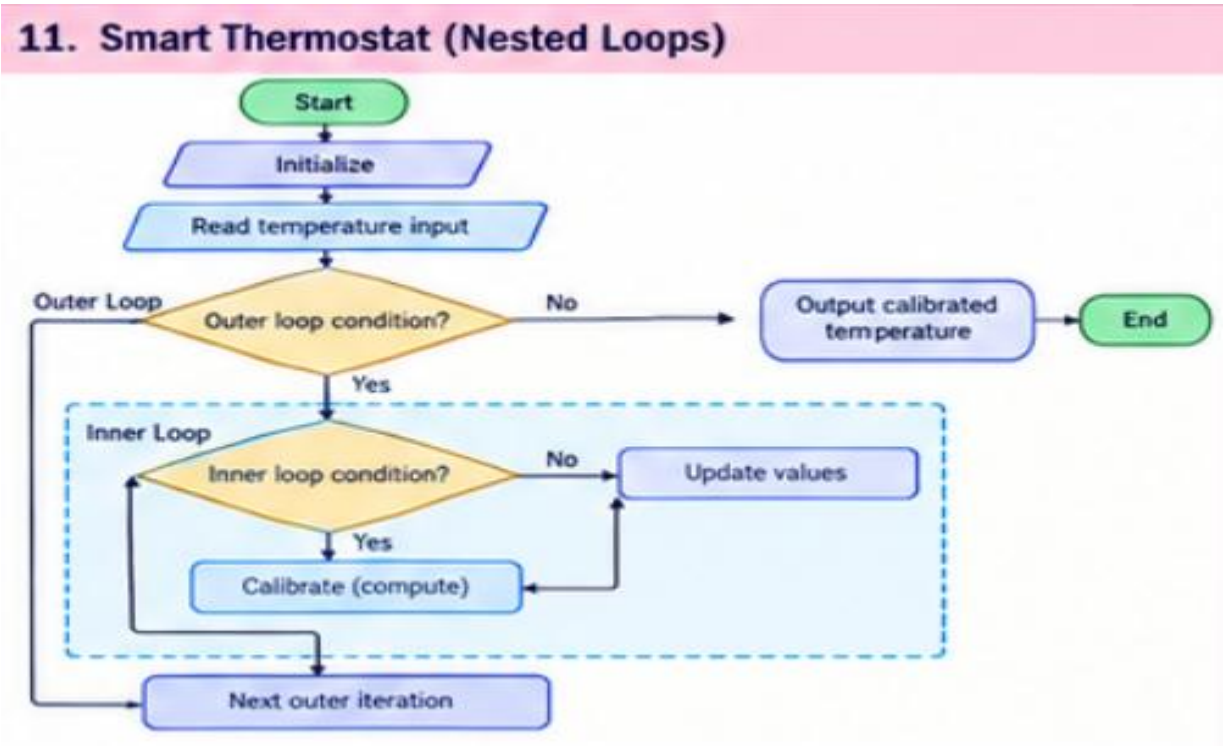
1. System Understanding

The firmware execution trace shows repeated nested loops during temperature calibration. Therefore, the testing strategy should concentrate on loop behaviour.

The calibration module can be decomposed into:

1. Temperature input
2. Sensor reading
3. Calibration calculation
4. Outer iteration
5. Inner iteration
6. Convergence/termination
7. Final calibrated output

2. Conceptual Structure



3. Loop Invariants

A loop invariant is a condition that remains valid during loop execution.

Examples:

- Calibration values remain within permitted range.
- Iteration count never exceeds the maximum.

- Temperature error gradually approaches the acceptable tolerance.

4. Boundary Loop Tests

For a loop expected to execute **N** times:

- 0 iterations
- 1 iteration
- 2 iterations
- N − 1
- N
- N + 1

For nested loops, these conditions should be tested independently and together.

5. Important Test Cases

TC	Condition	Expected
TC01	Outer loop = 0	Immediate termination
TC02	Inner loop = 0	Correct outer handling
TC03	One inner iteration	Correct result
TC04	Maximum iterations	Correct termination
TC05	Maximum + 1	Prevent excessive execution
TC06	Convergence reached early	Exit loop
TC07	No convergence	Safe termination

6. Preventing Firmware Hangs

The implementation should contain:

- Maximum iteration count
- Proper termination condition
- Valid loop counter
- Timeout mechanism
- Handling for non-converging calibration

Conclusion

Boundary-focused loop testing is essential because incorrect termination conditions can cause infinite loops and firmware hangs. Testing zero, one, maximum and excessive iterations provides strong loop coverage.

QUESTION 12

Reverse Engineering Tax Calculation DLL

1. System Understanding

The DLL accepts income values and produces error codes corresponding to different income ranges. This behaviour can be used to reconstruct hidden tax slabs.

The functional components are:

1. Income input
2. Input validation
3. Income classification
4. Tax-slab selection
5. Tax calculation
6. Error-code generation
7. Result output

2. Equivalence Classes

If the tax system has income slabs:

Income

|

├── Below Slab 1

├── Slab 1

├── Slab 2

├── Slab 3

└── Above Maximum

Each range becomes an equivalence class.

3. Boundary Value Analysis

For every slab boundary **B**, test:

- B − 1
- B
- B + 1

For example, if a boundary is ₹5,00,000:

- ₹4,99,999
- ₹5,00,000
- ₹5,00,001

4. Reconstructed Test Cases

TC	Income	Purpose
TC01	Negative income	Invalid input
TC02	0	Lower boundary
TC03	Boundary − 1	Previous slab
TC04	Boundary	Exact slab boundary
TC05	Boundary + 1	Next slab
TC06	Normal income in slab	Valid calculation
TC07	Very high income	Highest slab
TC08	Invalid/non-numeric input	Validation

5. Error-Code Validation

The observed error code for each income range should be compared with the reconstructed slab.

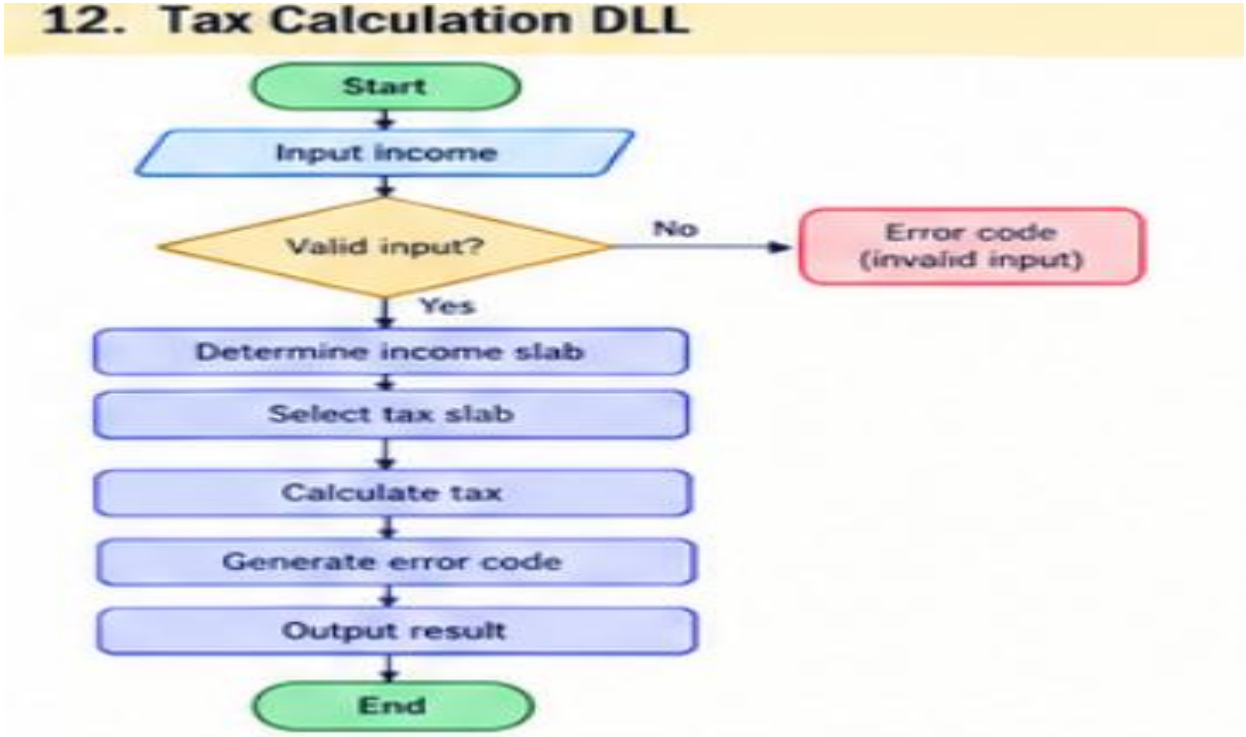
If:

Income ∈ Slab 1 → Expected Code 1

Income ∈ Slab 2 → Expected Code 2

Income ∈ Slab 3 → Expected Code 3

then boundary tests determine whether the DLL correctly changes from one category to another.



Conclusion

ECP identifies representative income ranges, while BVA verifies exact slab transitions. Together they provide strong evidence that the hidden tax-slab logic has been correctly reconstructed.

QUESTION 13

Hybrid Testing of Flight Booking System

1. System Understanding

The integration test reports show defects only when three conditions occur together:

- Frequent Flyer status
- International travel
- Last-minute booking

This indicates that individual conditions may have been tested, but their **combination** was missed.

2. Functional Decomposition

Flight Booking

```
|
├── Passenger Details
├── Frequent Flyer Validation
├── Travel Type
|   ├── Domestic
|   └── International
├── Booking Time
|   ├── Normal
|   └── Last-minute
├── Fare/Eligibility Rules
└── Booking Confirmation
```

3. Cause-Effect Analysis

Let:

- C1 = Frequent Flyer
- C2 = International
- C3 = Last-minute

The important defect condition is:

C1 AND C2 AND C3

↓

Special Rule

↓

Booking/Fare Decision

Therefore, the test suite must explicitly include the combination:

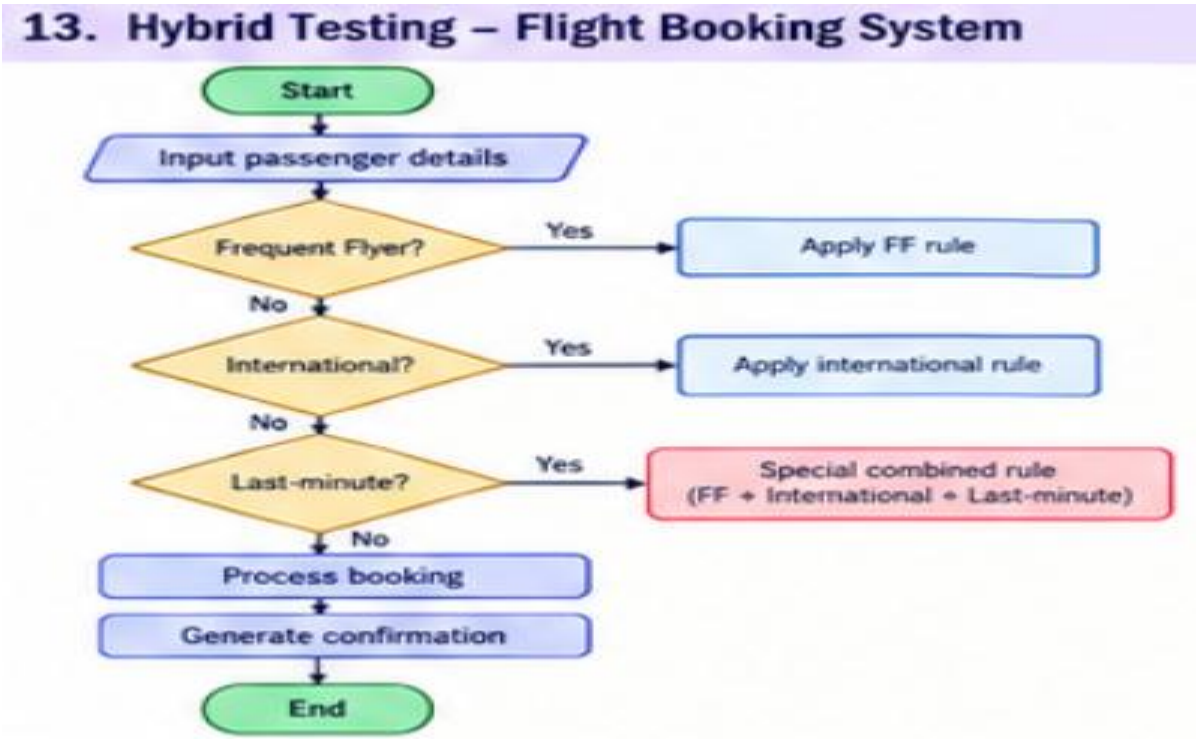
Frequent Flyer + International + Last-minute

4. Combination Test Cases

TC	Frequent Flyer	International	Last-minute	Expected
TC01	N	N	N	Normal booking
TC02	Y	N	N	Frequent flyer rule
TC03	N	Y	N	International rule
TC04	N	N	Y	Last-minute rule
TC05	Y	Y	N	Combined FF + international
TC06	Y	N	Y	FF + last-minute
TC07	N	Y	Y	International + last-minute
TC08	Y	Y	Y	Critical combined rule

5. White-Box Path Testing

The internal booking logic can be represented as:



Each decision should be tested for both True and False outcomes.

6. Hybrid Strategy

The reconstructed strategy combines:

Black-box testing

- Cause-Effect Graphing
- Equivalence Partitioning
- Combination testing

White-box testing

- Control Flow Graph
- Branch testing
- Path testing

7. Why the Original Testing Missed the Defect

The original testing probably tested each condition separately:

- Frequent Flyer + Domestic
- Normal customer + International
- Frequent Flyer + Last-minute

But it failed to test all three simultaneously.

Therefore, the defect existed in the interaction between business rules.

Conclusion

A hybrid Cause-Effect + Path Testing strategy provides better coverage for complex business workflows. Cause-Effect Graphing identifies important input combinations, while Path Testing ensures the corresponding internal decision paths are executed.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation